МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ

Ордена трудового Красного Знамени федеральное государственное бюджетное

образовательное учреждение высшего образования

«Московский технический университет связи и информатики»

Кафедра «Программная инженерия»

Отчет по лабораторной работе 4

Обработка исключений

Выполнила: студентка группы БПИ2401

Садовникова Евгения

Проверил: Харрасов Камиль Раисович

Москва, 2025

Содержание

Цель работы 3

Задания 3

Ход работы: 4

Задание 1 4

Листинг 1. class ArrayAverage 4

Выходные данные: 5

Задание 2 6

Листинг 2. class FileCopy 6

Выходные данные: 6

Задание 3 7

Листинг 3. class CustomUnsupportedOperationException 7

Листинг 4. class CustomMath 7

Листинг 5. class Main 8

Листинг 6. class ExceptionLogger 8

Выходные данные: 9

Контрольные вопросы 10

Вывод 15

Ссылка на GitHub: 15

Цель работы:

Изучить и применить механизмы обработки исключений в Java, включая перехват ошибок работы с массивами и файлами, создание собственных исключений, обработку неподдерживаемых операций и логирование возникающих ошибок

Задания

Задание 1

Написать программу, которая будет находить среднее арифметическое элементов массива. При этом программа должна обрабатывать ошибки, связанные с выходом за границы массива и неверными данными (например, если элемент массива не является числом). Пример реализации метода:

```
1.  public class ArrayAverage {
2.    public static void main(String[] args) {
3.      int[] arr = {1, 2, 3, 4, 5};
4.      int sum = 0;
5.      try {
6.          // Обработка массива
7.          }
8.          // Вывод массива
9.      } catch (*исключение*) {
10.        // Вывод исключения
11.     } catch (*исключение*) {
```

```
        // Вывод исключения
      }
    }
}
```

Задание 2

Написать программу, которая будет копировать содержимое одного файла в другой. При этом программа должна обрабатывать возможные ошибки, связанные:

• с открытием и закрытием файлов (вариант 1);

• чтением и записью файлов (вариант 2).

Задание 3

Создайте Java-проект для работы с исключениями. Для каждой из восьми указанных ниже задач напишите собственный класс для обработки исключений. Создайте обработчик исключений, который логирует информацию о каждом выброшенном исключении в текстовый файл.

8. Создайте класс CustomUnsupportedOperationException, который будет использоваться для обработки исключения UnsupportedOperationException. Реализуйте программу, которая выполняет математические операции (сложение, вычитание, умножение, деление) с помощью собственного класса и выбрасывайте исключение CustomUnsupportedOperationException, если операция не поддерживается.

Ход работы:

Задание 1

Создадим класс ArrayAverage с методом main, добавим туда обработку ошибок ArithmeticException, IllegalArgumentException, ArrayIndexOutOfBoundsException (листинг 1).

Листинг 1. class ArrayAverage

```
public class ArrayAverage {
    public static void main(String[] args) {
        int[] int_arr = { 1, 2, 3, 4, 4 };
        // String[] String_arr = { "1", "2", "3", "4", "5" };
        String[] String_arr = { "1", "2", "three", "4", "5" };
        int sum = 0;
        try {
            // Правильный
            for (int i = 0; i < int_arr.length; i++) {
                sum += int_arr[i];
            }
            double avg = (double) sum / int_arr.length;
            System.out.println("Array Average: " + avg);

            // ArithmeticException
            System.out.println(sum / 0);

            // IllegalArgumentException
            sum = 0;
            for (int i = 0; i < String_arr.length; i++) {
                sum += Integer.parseInt(String_arr[i]);
            }
```

```
                avg = (double) sum / String_arr.length;
                System.out.println("Array Average: " + avg);

                // ArrayIndexOutOfBoundsException
                sum = 0;
                for (int i = 0; i <= int_arr.length; i++) {
                    sum += int_arr[i];
                }
            } catch (ArrayIndexOutOfBoundsException e) {
                System.out.println("Error:\nGoing beyond the boundaries
of the array\n" + e.getMessage());
                // Выход за границы массива
            } catch (ArithmeticException e) {
                 System.out.println("Error:\nDivision by zero or other
arithmetic error\n" + e.getMessage());
                // Деление на ноль или другая арифметическая ошибка
            } catch (IllegalArgumentException e) {
                  System.out.println("Error:\nPassing an inappropriate
argument to a method\n" + e.getMessage());
                // передача неподобающего аргумента в метод
                 // (например, отрицательное, когда метод предполагает
задание положительных
                // значений)
            }
        }
```

Выходные данные:

Работа программы представлена на рисунке 1, где отображены различные обработки ошибок.

```
PS C:\Users\Evgen\Desktop\ИТИП> java ArrayAverage.java
Array Average: 2.8
Error:
Division by zero or other arithmetic error
/ by zero
PS C:\Users\Evgen\Desktop\ИТИП> java ArrayAverage.java
Array Average: 2.8
Error:
Passing an inappropriate argument to a method
For input string: "three"
PS C:\Users\Evgen\Desktop\ИТИП> java ArrayAverage.java
Array Average: 2.8
Array Average: 3.0
Error:
Going beyond the boundaries of the array
Index 5 out of bounds for length 5
PS C:\Users\Evgen\Desktop\ИТИП>
```

Рисунок 1. Результат программы

Задание 2

Создадим класс FileCopy с обработкой исключений IOException (листинг 2).

Листинг 2. class FileCopy

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;

public class FileCopy {
    public static void main(String[] args) {
        String sourceFile = "source.txt";
        String targetFile = "target.txt";

        try (FileInputStream fis = new FileInputStream(sourceFile);
                                    FileOutputStream   fos   =   new
FileOutputStream(targetFile)) {
            int data;
            try {
                while ((data = fis.read()) != -1) {
                    fos.write(data);
                }
                 System.out.println(sourceFile + " is copied to " +
targetFile);
            } catch (IOException e) {
                System.out.println("Error when reading or writing a
file:");// Ошибка при чтении или записи файла:
                System.out.println(e.getMessage());
            }
        } catch (IOException e) {
              System.out.println("Error when opening or closing a
file:");// Ошибка при открытии или закрытии файла:
            System.out.println(e.getMessage());
        }
    }
}
```

Выходные данные:

Работа программы представлена на рисунке 2, где отображены различные обработки ошибок.

```
PS C:\Users\Evgen\Desktop\ИТИП> java FileCopy.java
source.txt is copied to target.txt
PS C:\Users\Evgen\Desktop\ИТИП> java FileCopy.java
Error when opening or closing a file:
sourceP№.txt (Не удается найти указанный файл)
PS C:\Users\Evgen\Desktop\ИТИП> java FileCopy.java
Error when reading or writing a file:
Stream Closed
PS C:\Users\Evgen\Desktop\ИТИП>
```

Рисунок 2. Результат программы

Задание 3

Создадим класс CustomUnsupportedOperationException, который будет использоваться для обработки исключения UnsupportedOperationException.(листинг 3)

Реализуем программу, которая выполняет математические операции (сложение, вычитание, умножение, деление) с помощью собственного класса и выбрасывайте исключение CustomUnsupportedOperationException, если операция не поддерживается. (листинг 4–5)

Создадим обработчик исключений, который логирует информацию о каждом выброшенном исключении в текстовый файл.(листинг 6)

Листинг 3. class CustomUnsupportedOperationException

```
public class CustomUnsupportedOperationException extends UnsupportedOperationException {
    public CustomUnsupportedOperationException() {
    }

    public CustomUnsupportedOperationException(String str) {
        super(str);
    }
}
```

Листинг 4. class CustomMath

```
public class CustomMath {
    public static double calculate(double a, double b, String operation)
                    throws CustomUnsupportedOperationException, ArithmeticException {
        if ("+".equals(operation)) {
            return a + b;
```

```
            } else if ("-".equals(operation)) {
                return a - b;
            } else if ("*".equals(operation)) {
                return a * b;
            } else if ("/".equals(operation)) {
                if (b == 0) {
                   throw new ArithmeticException("Division by zero!");//
Деление на ноль
                }
                return a / b;
            } else {
               throw new CustomUnsupportedOperationException("Operation
'" + operation + "' not supported");
                // Операция не поддерживается
            }
        }

    }
```

Листинг 5. class Main

```
    public class Main {
        public static void main(String[] args) {
            double a = 1.0;
            double b = 2.0;
            try {
                System.out.println(CustomMath.calculate(a, b, "+"));
                System.out.println(CustomMath.calculate(a, b, "-"));
                System.out.println(CustomMath.calculate(a, b, "*"));
                System.out.println(CustomMath.calculate(a, b, "/"));
                System.out.println(CustomMath.calculate(a, b, "^"));
            } catch (CustomUnsupportedOperationException e) {
                System.out.println(e.getMessage());
                ExceptionLogger.log(e);

            } catch (ArithmeticException e) {
                System.out.println(e.getMessage());
                ExceptionLogger.log(e);

            } catch (Exception e) {
                System.out.println(e.getMessage());
                ExceptionLogger.log(e);
            }

        }
    }
```

Листинг 6. class ExceptionLogger

```
    import java.io.FileWriter;
    import java.io.IOException;
    import java.time.LocalDateTime;

    public class ExceptionLogger {

        private static final String LOG_FILE = "errors.log";
```

```
        // Метод логирования исключения в файл
        public static void log(Exception e) {
            try (FileWriter fw = new FileWriter(LOG_FILE, true)) {
                fw.write("=== ERROR ===\n");
                fw.write("Timestamp: " + LocalDateTime.now() + "\n");
                 fw.write("Exception Type: " + e.getClass().getName() +
"\n");
                fw.write("Message: " + e.getMessage() + "\n\n");
            } catch (IOException io) {
                System.out.println("Error when writing to the log file:
" + io.getMessage());
                // Ошибка при записи в лог-файл:
            }
        }
    }
```

Выходные данные:

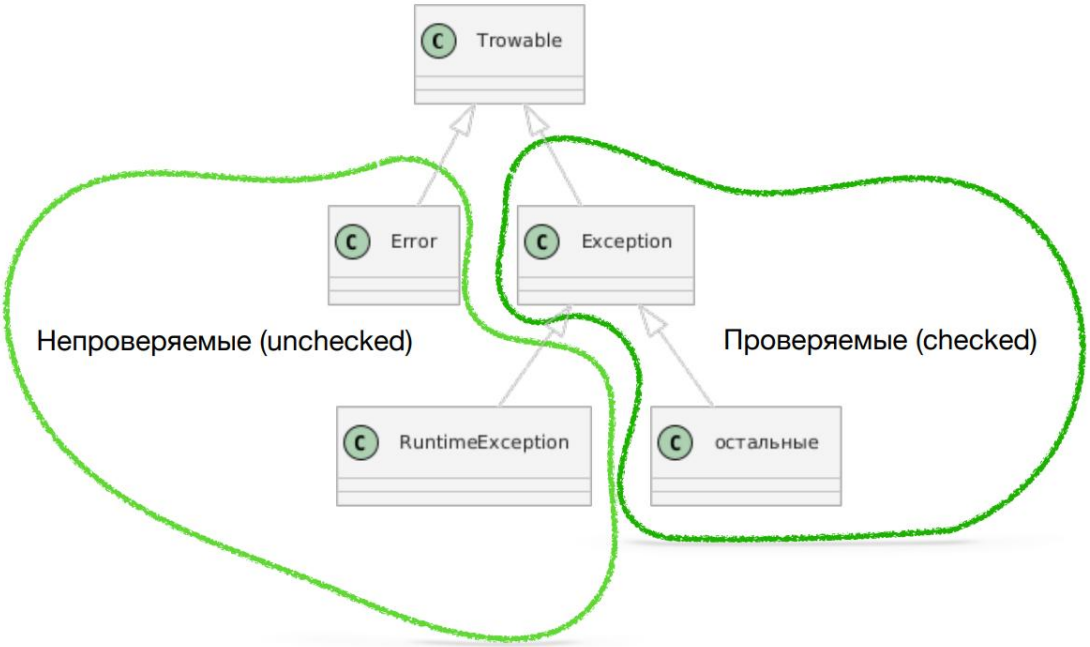
```
errors.log
1   === ERROR ===
2   Timestamp: 2025-12-01T01:59:15.882883100
3   Exception Type: CustomUnsupportedOperationException
4   Message: Operation '^' not supported
5
6   === ERROR ===
7   Timestamp: 2025-12-01T02:00:52.417141400
8   Exception Type: CustomUnsupportedOperationException
9   Message: Operation '^' not supported
10
11  === ERROR ===
12  Timestamp: 2025-12-01T02:02:35.127401900
13  Exception Type: CustomUnsupportedOperationException
14  Message: Operation '^' not supported
15
16  === ERROR ===
17  Timestamp: 2025-12-01T02:02:40.195653900
18  Exception Type: CustomUnsupportedOperationException
19  Message: Operation '^' not supported
20
21
PROBLEMS 6   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS C:\Users\Evgen\Desktop\ИТИП> java Main
3.0
-1.0
2.0
0.5
Operation '^' not supported
PS C:\Users\Evgen\Desktop\ИТИП>
```

Контрольные вопросы

1. Что такое исключение в Java?

Исключения в Java являются механизмом обработки ошибок и нестандартных ситуаций во время выполнения программы. Они позволяют изолировать ошибки и обрабатывать их в специальном блоке кода, что делает программу более надежной и устойчивой к ошибкам.

2. Какие ключевые классы исключений вы знаете?



Все исключения в Java являются потомками класса Throwable. От него наследуются два класса:

- Error - это ошибки, возникающие при выполнении программы в результате сбоя работы JVM, переполнения памяти или сбоя системы. Обычно они свидетельствуют о серьезных проблемах, устранить которые программными средствами невозможно. Такой вид исключений в Java относится к непроверяемым (unchecked) на стадии компиляции;
- Exception - обычные ошибки, которые возникают во время работы многих методов, например ошибка открытия файла

Среди наследников класса Exception есть класс RuntimeException - это суперкласс тех исключений, которые могут быть выброшены во время

нормальной работы JVM, например IndexOutOfBoundsException - когда индекс находится вне пределов массива

3. Что такое проверяемые и непроверяемые исключения?

Класс Exception и все его подклассы, которые не являются также подклассами RuntimeException, являются проверяемыми исключениями. Проверяемые исключения должны быть задекларированы в заголовке метода при помощи ключевого слова throws, если они могут быть выброшены при выполнении метода или конструктора и распространиться за пределы границы метода или конструктора или обработаны в том методе, где они могут возникнуть.

Декларация исключения в методе в Java означает указание типов исключений, которые данный метод может генерировать во время своего выполнения.

```
void methodName() throws ExceptionType1,
        ExceptionType2, ... {
    // тело метода
}
```

Error, RuntimeException и его подклассы являются непроверяемыми исключениями. Непроверяемые исключения не нужно объявлять в поле throws метода или конструктора, если они могут быть выброшены при выполнении метода или конструктора и распространяться за пределы границ метода или конструктора.

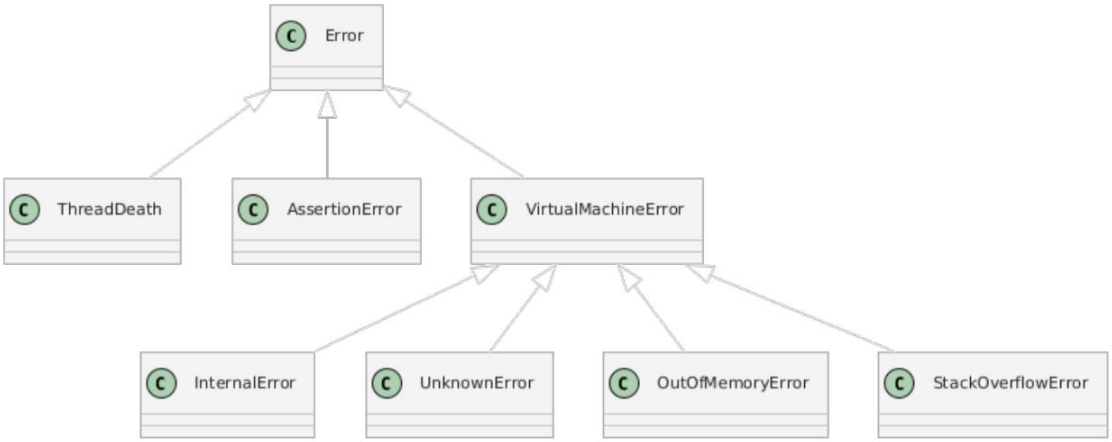
4. Какие исключения и как необходимо обрабатывать?

Проверяемые исключения нужно обязательно обрабатывать через try-catch или пробрасывать с помощью throws.

Непроверяемые исключения можно обрабатывать при необходимости для повышения устойчивости программы.

5. Какие исключения относятся к классу Error и как их обрабатывать?

Error - серьёзные ошибки виртуальной машины, например StackOverflowError или OutOfMemoryError. Обычно их не обрабатывают, программа может завершиться аварийно.



6. Какие исключения относятся к классу RuntimeException и как их обрабатывать?

RuntimeException - ошибки времени выполнения (NullPointerException, ArrayIndexOutOfBoundsException). Их обработка через try-catch рекомендуется, если возможны ошибки, влияющие на работу программы.

7. Как создать собственный класс исключения?

Создаётся класс, наследующий Exception или его подклассы, с конструктором, вызывающим super(message).

```
public class CustomException extends Exception {
    public CustomException(String message) {
        super(message);
    }
}
```

8. Как обрабатываются исключения в Java? Какие конструкции для этого используются?

try - блок кода, где может возникнуть исключение.

catch - блок обработки исключения.

finally - блок, выполняющийся всегда.

try-with-resources - автоматическое закрытие ресурсов.

9. Можно ли использовать try без catch или finally?

Нет. try должен сопровождаться хотя бы одним блоком catch или finally.

10. Что произойдет, если исключение возникнет в блоке finally?

Исключение, возникшее в finally, заменяет любое исключение из блоков try или catch и передаётся дальше по стеку вызовов.

11. Как можно пробросить исключение выше по стеку вызовов?

С помощью ключевого слова throws в объявлении метода:

```
public void readFile() throws IOException {
    FileReader reader = new FileReader("file.txt");
    // Если файл не найден, IOException будет проброшено выше
}

public void process() {
    try {
        readFile();
    } catch (IOException e) {
        System.out.println("Ошибка при чтении файла: " +
e.getMessage());
    }
}
```

12.В чем разница между finally и try-with-resources?

finally выполняется всегда, используется для ручного закрытия ресурсов.

try-with-resources автоматически закрывает ресурсы, реализующие AutoCloseable.

13. Какие классы можно использовать в try-with-resources? Что такое AutoCloseable?

В try-with-resources можно использовать любой класс, реализующий интерфейс AutoCloseable или его наследника Closeable.

Интерфейс AutoCloseable содержит метод close(), который вызывается автоматически после завершения блока try.

14. Можно ли в одном try использовать несколько catch блоков? В каком порядке их нужно располагать?

Да. Сначала пишут более конкретные исключения, затем более общие, иначе компилятор выдаст ошибку.

```
 try {
   // Код, который может вызвать исключение
} catch (ExceptionType1 e) {
   // Обработка исключения типа ExceptionType1
} catch (ExceptionType2 e) {
   // Обработка исключения типа ExceptionType2
} finally {
   // Код, который выполняется всегда, независимо от того,
   // возникло ли исключение или нет
}
```

15. В чем разница между throw и throws?

throw - выброс конкретного исключения в коде.

throws - объявление метода о том, что он может выбросить исключение.

16. Что такое StackOverflowError и OutOfMemoryError? Можно ли их обработать?

StackOverflowError - переполнение стека вызовов, например из-за бесконечной рекурсии.

OutOfMemoryError - недостаток памяти.

Обрабатывать их можно через try-catch(Error e), но обычно их не обрабатывают, так как это критические ошибки JVM.

Вывод

В ходе лабораторной работы были изучены и применены механизмы обработки исключений в Java. Были реализованы программы с перехватом ошибок при работе с массивами и файлами, создан собственный класс исключений CustomUnsupportedOperationException для обработки неподдерживаемых операций, а также реализован логгер, записывающий информацию о возникающих исключениях в текстовый файл.

Ссылка на GitHub: https://github.com/Evgeshhha/Information-technology-and-programming